Bahasa Pemrograman Tingkat Rendah (MWCA010)

Program Studi Teknik Informatika ▪ Fakultas Teknologi Informasi ▪ Universitas Bale Bandung

Instruksi Transfer Data dan Aritmatika

Pertemuan 9

Dosen Pengampu

Cecep Suwanda, S.Si., M.Kom.

Semester Ganjil

Capaian dan Tujuan Pembelajaran

Sub-CPMK 3.1

Menulis program assembly lengkap dengan instruksi transfer data (`MOV`, `XCHG`, `LEA`, `PUSH`/`POP`) dan instruksi aritmatika (`ADD`, `SUB`, `INC`, `DEC`, `MUL`, `IMUL`, `DIV`, `IDIV`).

CPMK yang didukung: CPMK03 ▪ **CPL yang didukung: CPL03**

Setelah pertemuan ini, mahasiswa mampu:

1. menerapkan aturan legalitas `MOV` dan mengatasi pemindahan yang ditolak dengan register perantara;
2. membedakan `MOV` dan `LEA`: nilai data atau alamat data;
3. memakai `PUSH` dan `POP` untuk menyimpan dan mengambil kembali data pada *stack*;
4. menjelaskan pengaruh aritmatika terhadap `CF`, `ZF`, `SF`, `OF`, `AF`, dan `PF`;
5. menuliskan perkalian dan pembagian dengan operand implisit yang benar, menangani *divide overflow*, dan menyusun aritmatika bertingkat dengan `ADC` dan `SBB`.

Peta Materi

1. **Instruksi Transfer Data**
   `MOV` dan tiga aturan legalitasnya; register perantara; `XCHG`; perbedaan `MOV` dan `LEA`; `PUSH`/`POP`; `LDS`/`LES`

2. **Instruksi Aritmatika Dasar dan Pengaruhnya pada *Flag***
   `ADD` dan `ADC`; `SUB`, `SBB`, `INC`, `DEC`, `NEG`; makna *flag*; operasi bertanda dan tak bertanda; `CMP`

3. **Perkalian, Pembagian, dan Aritmatika Bertingkat**
   operand implisit; *divide overflow*; `IMUL`/`IDIV` serta `CBW`/`CWD`; `ADC` dan `SBB` untuk lebih dari 16 bit; konversi ke digit desimal

Praktik pada pertemuan ini

Menelusuri `aritmatika.asm` dan `kali-bagi.asm`, mencatat nilai *flag*, lalu membangun kalkulator empat operasi sebagai Tugas 4.

BAGIAN

Instruksi Transfer Data

Instruksi MOV: Menyalin, Bukan Memindahkan

`MOV tujuan, sumber` menyalin data ke tujuan; isi sumber *tidak* terhapus. Setelah `MOV AX, BX`, kedua register berisi nilai yang sama.

Tiga aturan legalitas operand

1. **Ukuran kedua operand harus sama.** `MOV AX, BL` ditolak karena `AX` 16 bit sedangkan `BL` 8 bit.
2. **Tidak boleh memori ke memori.** `MOV nilai2, nilai1` ditolak; 8086 tidak memiliki jalur antarlokasi memori.
3. **Tidak boleh nilai segera ke register segmen.** `MOV DS, 1000h` ditolak; harus melalui register umum.

Jebakan yang sering terjadi

Menambal `MOV nilai2, nilai1` dengan `WORD PTR` tidak menyelesaikan masalah — penyebabnya jenis operand, bukan ukurannya.

Sumber: Bab X, Instruksi MOV dan Aturan Legalitasnya

Register Perantara: Cara Memperbaiki yang Ditolak

Setiap kombinasi yang ditolak diselesaikan dengan menyisipkan register perantara, bukan dengan menambah PTR.

```
; memori ke memori : dua langkah melalui AX
MOV AX, nilai1          ; muat isi memori ke AX
MOV nilai2, AX          ; simpan dari AX ke memori tujuan

; nilai segera ke DS : dua langkah melalui AX
MOV AX, 1000h           ; muat nilai segera ke AX
MOV DS, AX              ; salin dari AX ke DS
```

Pola dua langkah yang sama juga dipakai untuk pemindahan segmen ke segmen: `MOV AX, ES` lalu `MOV DS, AX`.

Sumber: Bab X, Mengatasi Pemindahan yang Tidak Diizinkan

Tabel Legalitas Kombinasi Operand `MOV`

Bentuk pemindahan	Contoh	Sah	Keterangan
Register ke register	`MOV AX, BX`	Ya	Tanpa akses memori
Nilai segera ke register	`MOV AX, 5`	Ya	Nilai segera muat 16 bit
Memori ke register	`MOV AX, nilai1`	Ya	Satu operand di memori
Register ke memori	`MOV nilai1, AX`	Ya	Menyimpan hasil hitungan
Register ke segmen	`MOV DS, AX`	Ya	Cara baku inisialisasi `DS`
Nilai segera ke segmen	`MOV DS, 1000h`	Tidak	Harus lewat register umum
Memori ke memori	`MOV nilai2, nilai1`	Tidak	Pakai register perantara

Sumber: Bab X, Tabel Legalitas Kombinasi Operand pada Instruksi MOV

Instruksi XCHG: Menukar Dua Operand

XCHG menukar isi dua operand sehingga keduanya berubah sekaligus, tanpa memerlukan register ketiga.

```
MOV AX, 1234h        ; AX = 1234h
MOV BX, 5678h        ; BX = 5678h
XCHG AX, BX          ; AX = 5678h, BX = 1234h
```

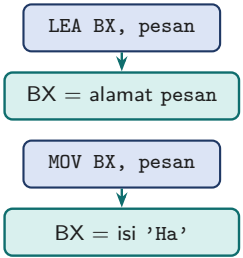
Aturan XCHG lebih ketat daripada MOV

Minimal satu operand wajib berada di register, kedua operand harus berukuran sama, dan tidak ada operand yang boleh berupa nilai segera.

Sumber: Bab X, Instruksi XCHG

Perbedaan MOV dan LEA

LEA memuat *alamat* operand, sedangkan MOV memuat *isi* operand.

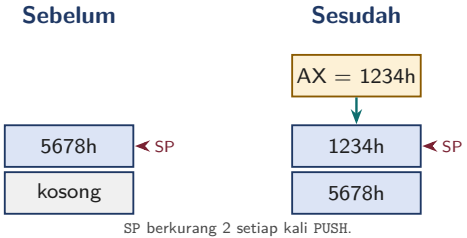



```
pesan DB 'Halo$'

    LEA BX, pesan      ; BX = alamat pesan
    MOV BX, OFFSET pesan
                       ; hasilnya sama
    MOV BX, pesan      ; BX = isi 'Ha'
```

Ketiganya sah secara sintaksis, sehingga assembler tidak melaporkan kesalahan — programnya saja yang salah.

Instruksi PUSH dan POP pada *Stack*



Aturan pemakaian

- Bekerja pada satuan word; `PUSH AL` tidak tersedia.
- Nilai segera tidak diizinkan sebagai operand.
- `CS` boleh di-`PUSH`, tidak boleh menjadi tujuan `POP`.
- Setiap `PUSH` harus berpasangan dengan `POP` yang seimbang agar `SP` kembali ke posisi semula.

Sumber: Bab X, Instruksi PUSH dan POP

Instruksi LDS, LES, dan Ringkasan Transfer Data

Instruksi	Bentuk umum	Kegunaan	Contoh
MOV	`MOV tujuan, sumber`	Menyalin data; sumber tetap	`MOV AX, BX`
XCHG	`XCHG op1, op2`	Menukar isi dua operand	`XCHG AX, BX`
LEA	`LEA reg16, mem`	Memuat alamat efektif	`LEA SI, pesan`
LDS	`LDS reg16, mem32`	Offset ke reg, segmen ke DS	`LDS SI, ptrJauh`
LES	`LES reg16, mem32`	Offset ke reg, segmen ke ES	`LES DI, ptrJauh`
PUSH	`PUSH sumber`	Menaruh word ke *stack*	`PUSH AX`
POP	`POP tujuan`	Mengambil word dari *stack*	`POP BX`

Sumber: Bab X, Instruksi LDS dan LES serta Tabel Ringkasan Transfer Data

Rangkuman Bagian 1

- `MOV` menyalin data tanpa menghapus sumber. Legalitasnya dibatasi tiga aturan: ukuran operand harus sama, tidak boleh memori ke memori, dan tidak boleh nilai segera ke register segmen.
- Pemindahan yang ditolak diselesaikan dengan register perantara dua langkah, bukan dengan menambahkan `WORD PTR`.
- `XCHG` menukar isi dua operand dan mensyaratkan minimal satu operand berada di register; nilai segera tidak diizinkan.
- `MOV` memindahkan *isi* memori, sedangkan `LEA` memindahkan *alamat* memori — lebih ringkas dan tidak menyentuh memori bila alamat dihitung saat program berjalan.
- `PUSH` dan `POP` bekerja pada satuan word, tidak menerima nilai segera, dan harus berpasangan seimbang agar `SP` kembali ke posisi semula.

BAGIAN

Instruksi Aritmatika Dasar dan Pengaruhnya pada *Flag*

Penjumlahan: ADD dan ADC

ADD menjumlahkan sumber ke tujuan dan menyimpan hasilnya pada tujuan. ADC bekerja sama, tetapi ikut menambahkan CF dari operasi sebelumnya.

```
MOV AX, 25              ; AX = 25
ADD AX, 17              ; AX = 42
ADD nilai1, AX          ; nilai1 = nilai1 + AX (di memori)
ADC AX, BX              ; AX = AX + BX + CF
```

Kapan perbedaannya terasa

Pada operasi satu langkah keduanya sering dianggap sama karena CF biasanya nol. Bedanya baru tampak pada aritmatika bertingkat, yaitu saat *carry* harus diteruskan ke word berikutnya.

Sumber: Bab X, Penjumlahan ADD dan ADC

Pengurangan dan Penyangkalan

SUB mengurangi tujuan dengan sumber, SBB ikut mengurangkan CF, INC dan DEC menambah atau mengurangi satu, dan NEG mengubah tanda bilangan.

```
MOV AX, 100             ; AX = 100
SUB AX, 45              ; AX = 55
INC AX                  ; AX = 56
DEC AX                  ; AX = 55
NEG AX                  ; AX = -55 (0FFC9h)
SBB AX, BX              ; AX = AX - BX - CF
```

Dua kesalahan yang sering terjadi

NEG tidak selalu menyetel CF: bila AX nol, hasilnya nol dan CF bernilai 0. INC sengaja tidak mengubah CF, sehingga tidak boleh dipakai pada aritmatika bertingkat.

Sumber: Bab X, Pengurangan dan Penyangkalan

Perbandingan dengan CMP

CMP bekerja identik dengan SUB, tetapi *tidak menyimpan* hasilnya. Yang ditinggalkan hanyalah jejak pada *flag*, sehingga CMP selalu diikuti lompatan bersyarat.

```
CMP AX, BX          ; AX - BX
JE  SAMA            ; ZF = 1
JA  LEBIH_BESAR     ; CF = 0, ZF = 0
JG  LEBIH_TTD       ; SF = OF, ZF = 0
```

Tiga pola yang paling sering

JE untuk kesamaan; JA untuk tak bertanda; JG untuk bertanda. Pilihan lompatan harus sejalan dengan tafsiran operandnya.

Karena hasilnya dibuang, CMP tidak mengubah nilai register apa pun — hanya *flag*.

Sumber: Bab X, Operasi Perbandingan dengan CMP

Makna *Flag* Hasil Aritmatika

Flag	Arti	Tanda pada hasil
`CF`	*Carry*: ada nilai yang keluar dari bit paling kiri; pada pengurangan berarti pinjaman	`CF` = 1 menandakan hasil tak bertanda melampaui kapasitas
`ZF`	*Zero*: hasil operasi bernilai nol	`ZF` = 1 menandakan kedua operand sama besar
`SF`	*Sign*: bit paling kiri hasil bernilai 1	`SF` = 1 menandakan hasil bertanda negatif
`OF`	*Overflow*: hasil bertanda melampaui batas yang dapat ditampung	`OF` = 1 menandakan luapan bilangan bertanda
`AF`	*Auxiliary carry* dari bit ke-3 menuju bit ke-4	Dipakai instruksi penyesuaian BCD seperti `DAA`
`PF`	*Parity*: cacah bit 1 pada byte paling rendah genap	Dipakai pada pemeriksaan kesalahan transmisi

Sumber: Bab X, Makna Flag Hasil Aritmatika

Ringkasan Instruksi Aritmatika dan *Flag*-nya

Instruksi	Bentuk operand	Hasil	Flag terpengaruh
ADD	ADD tujuan, sumber	tujuan ← tujuan + sumber	OF SF ZF AF PF CF
ADC	ADC tujuan, sumber	tujuan ← tujuan + sumber + CF	seluruh *flag* aritmatika
SUB	SUB tujuan, sumber	tujuan ← tujuan − sumber	seluruh *flag* aritmatika
SBB	SBB tujuan, sumber	tujuan ← tujuan − sumber − CF	seluruh *flag* aritmatika
INC	INC tujuan	tujuan ← tujuan + 1	OF SF ZF AF PF; CF tetap
DEC	DEC tujuan	tujuan ← tujuan − 1	OF SF ZF AF PF; CF tetap
NEG	NEG tujuan	tujuan ← 0 − tujuan	seluruh *flag*; CF = 1 bila hasil tidak nol
CMP	CMP tujuan, sumber	hasil dibuang, hanya *flag* yang diubah	sama dengan SUB

Sumber: Bab X, Ringkasan Instruksi Aritmatika 8086

Operasi Bertanda dan Tak Bertanda

Satu deretan bit yang sama dapat ditafsirkan sebagai bilangan tak bertanda atau bertanda dalam komplemen dua. Byte `FFh` berarti 255 bila tak bertanda, tetapi -1 bila bertanda.

```
; tak bertanda : 200 + 100
MOV AL, 200      ; AL = C8h
ADD AL, 100      ; AL = 2Ch, CF = 1
                 ; -> salah

; bertanda : -56 + 100
MOV AL, 200      ; AL = C8h = -56
ADD AL, 100      ; AL = 2Ch, OF = 0
                 ; -> benar
```

Aturan praktis

`CF` untuk hasil tak bertanda dan `OF` untuk hasil bertanda. Karena itu, pakai lompatan tak bertanda (`JA`, `JB`) setelah operasi tak bertanda, dan lompatan bertanda (`JG`, `JL`) setelah operasi bertanda.

Perbedaan yang sama mendasari pemilihan `MUL` atau `IMUL`.

Rangkuman Bagian 2

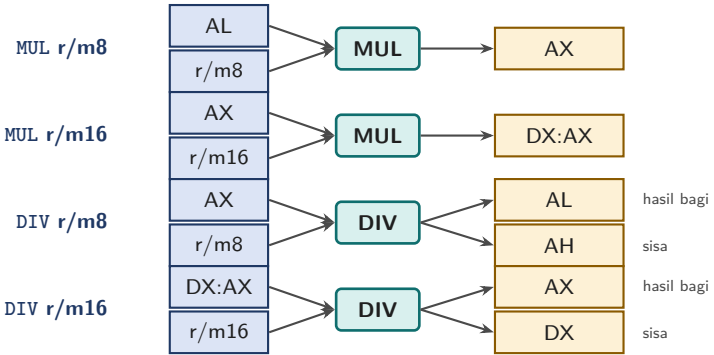
- Instruksi aritmatika dasar: `ADD`, `ADC`, `SUB`, `SBB`, `INC`, `DEC`, `NEG`, dan `CMP`.
- Setiap instruksi meninggalkan jejak pada *flag*, kecuali `CF` yang tidak disentuh `INC` dan `DEC` — itulah sebabnya keduanya tidak dipakai pada aritmatika bertingkat.
- `CF` menandai luapan tak bertanda, `OF` menandai luapan bertanda. Pilihan instruksi dan lompatan bersyarat harus mengikuti tafsiran yang dipilih.
- `NEG` menghitung $0 - AX$, sehingga `CF` bernilai 0 ketika hasilnya nol.
- `CMP` tidak menyimpan hasil, hanya menyetel *flag* sama seperti `SUB`, lalu diikuti `JE`, `JA`, atau `JG`.

BAGIAN

Perkalian, Pembagian, dan Aritmatika Bertingkat

Aturan Operand Implisit

`MUL`, `IMUL`, `DIV`, dan `IDIV` hanya menuliskan satu operand. Operand satunya sudah ditentukan prosesor dan tidak ditulis di dalam instruksi.



Sumber: Bab X, Aturan Operand Implisit

Operand Implisit dan Ukuran Hasil

Instruksi	Operand	Yang dihitung	Letak hasil
`MUL`	r/m8	`AL` × operand	`AX`; `CF` = `OF` = 1 bila `AH` tidak nol
`MUL`	r/m16	`AX` × operand	`DX`:`AX`; `CF` = `OF` = 1 bila `DX` tidak nol
`IMUL`	r/m8	`AL` × operand (bertanda)	`AX`; `OF` = 1 bila hasil tidak muat di `AL`
`IMUL`	r/m16	`AX` × operand (bertanda)	`DX`:`AX`; `OF` = 1 bila tidak muat di `AX`
`DIV`	r/m8	`AX` ÷ operand	hasil bagi di `AL`, sisa di `AH`
`DIV`	r/m16	`DX`:`AX` ÷ operand	hasil bagi di `AX`, sisa di `DX`

Hasil perkalian selalu berukuran dua kali operand, sehingga `MUL` tidak pernah kehilangan data; yang ada hanyalah penanda `CF` dan `OF`.

Sumber: Bab X, Operand Implisit dan Ukuran Hasil

Kesalahan *Divide Overflow*

Pembagian berperilaku sebaliknya: hasil bagi harus muat pada register tujuan yang ukurannya hanya separuh operand. Bila tidak muat, prosesor membangkitkan kesalahan *divide overflow* dan program DOS umumnya berhenti mendadak.

```
    MOV AX, 1000     ; AX = 03E8h
    MOV BL, 7        ; BL = pembagi
    CMP AH, BL       ; luapan bila AH >= BL
    JAE LUAPAN       ; periksa sebelum DIV
    DIV BL           ; AL = 142, AH = 6

LUAPAN:
    ...              ; laporkan, jangan DIV
```

Syarat aman

`DIV BL` aman hanya bila `AH` lebih kecil daripada `BL`. Untuk pembagian 16 bit, syaratnya `DX` lebih kecil daripada operand pembagi.

Luapan pembagian bukan kesalahan sintaks, sehingga tidak dilaporkan assembler.

Sumber: Bab X, Kesalahan Divide Overflow

Perkalian dan Pembagian Bertanda

IMUL dan IDIV menafsirkan operand sebagai bilangan bertanda dalam komplemen dua. Sebelum IDIV, bilangan yang dibagi harus diperluas tandanya.

```
MOV AX, 1000     ; AX = 03E8h
MOV BX, 7        ; BX = 0007h
CWD              ; DX = 0000h
IDIV BX          ; AX = 142, DX = 6
```

CBW dan CWD

CBW memperluas tanda AL ke AX; CWD memperluas tanda AX ke DX:AX.

Kesalahan karena CWD dihilangkan tidak terlihat pada bilangan positif; pada bilangan negatif, DX seharusnya bernilai FFFFh agar tafsiran bertanda menjadi benar.

Sumber: Bab X, Perkalian dan Pembagian Bertanda

Aritmatika Bertingkat dengan ADC dan SBB

Bilangan 32 bit dipecah menjadi word rendah (bit 0–15) dan word tinggi (bit 16–31). Penjumlahan dimulai dari word rendah; word tinggi memakai ADC agar *carry* ikut diperhitungkan.

```
MOV AX, rendahA      ; word rendah A
ADD AX, rendahB      ; AX = 1356h, CF = 1
MOV hasilL, AX

MOV AX, tinggiA      ; word tinggi A
ADC AX, tinggiB      ; AX = 8ACFh
MOV hasilT, AX
```

Propagasi *carry*

Word rendah: CDEFh + 4567h = 1356h, maka CF = 1.

Word tinggi: 89ABh + 0123h + 1 = 8ACFh.

Hasil akhir: 8ACF1356h.

Bila instruksi kedua ditulis ADD, hasilnya 8ACEh dan satu nilai hilang. Pengurangan 32 bit memakai pola SUB dan SBB.

Sumber: Bab X, Aritmatika Bertingkat dengan ADC dan SBB

Mengubah Bilangan Biner Menjadi Digit Desimal

Nilai di register selalu biner, sedangkan `INT 21h` fungsi `02h` mencetak satu karakter pada satu waktu. Bilangan harus dipecah menjadi digit dengan pembagian berulang sepuluh.

```
    MOV BX, 10          ; pembagi desimal
BAGI_DIGIT:
    XOR DX, DX          ; siapkan DX:AX
    DIV BX              ; AX bagi, DX sisa
    PUSH DX             ; simpan digit
    INC CX
    CMP AX, 0
    JNE BAGI_DIGIT
CETAK_DIGIT:
    POP DX
    ADD DL, 30h         ; nilai jadi ASCII
    MOV AH, 02h
    INT 21h
    LOOP CETAK_DIGIT
```

Alur algoritma

1. Bagi nilai dengan 10; sisanya satu digit.
2. Simpan sisa pada *stack*, tambah cacah digit.
3. Ulangi selama hasil bagi masih lebih besar dari nol.
4. Ambil kembali dari *stack* agar urutan cetak terbalik menjadi benar.
5. Tambahkan `30h` agar nilai 0–9 menjadi kode ASCII `'0'`–`'9'`.

Sumber: Bab X, Mengubah Bilangan Biner Menjadi Digit Desimal

Studi Kasus: Kalkulator Empat Operasi Sederhana

Kerangka program kalkulator mencakup tiga hal sekaligus: masukan pengguna, pemilihan operasi, dan pencetakan hasil.

1. Tampilkan menu: `1` tambah, `2` kurang, `3` kali, `4` bagi.
2. Baca pilihan, lalu arahkan alur program dengan `CMP` dan lompatan bersyarat.
3. Baca dua bilangan satu digit.
4. Jalankan operasi yang dipilih: `ADD`, `SUB`, `MUL`, atau `DIV`.
5. Untuk pembagian, periksa pembagi nol dan hasil bagi yang tidak muat sebelum `DIV` dijalankan.
6. Cetak hasil dengan prosedur konversi desimal `CETAK_AX`.

Kerangka ini menjadi dasar Tugas 4 pada bagian praktikum.

Sumber: Bab X, Studi Kasus Kalkulator Empat Operasi

Rangkuman Bagian 3

- `MUL`, `IMUL`, `DIV`, dan `IDIV` memakai operand implisit `AL` atau `AX`, dan menaruh hasil pada `AX` atau `DX:AX`.
- Hasil perkalian selalu dua kali ukuran operand, sehingga `MUL` tidak pernah kehilangan data; yang muncul hanya `CF` dan `OF`.
- `DIV` menuntut hasil bagi muat pada register tujuan. Syarat amannya `AH` $<$ `BL`, diperiksa dengan `CMP AH, BL` sebelum `DIV BL` dijalankan.
- Sebelum `IDIV`, tanda harus diperluas dengan `CBW` atau `CWD`; luapan pembagian tidak terdeteksi saat perakitan.
- Aritmatika bertingkat memakai `ADC` dan `SBB` agar *carry* dan pinjaman diteruskan dari word rendah ke word tinggi; konversi desimal memakai pembagian berulang sepuluh.

Contoh 1: Kalkulator Penjumlahan dan Pengurangan

```
    MOV AH, 01h         ; baca satu digit
    INT 21h
    SUB AL, 30h         ; ASCII -> nilai 0-9
    XOR AH, AH          ; nolkan AH
    MOV nilai1, AX      ; simpan bilangan 1
    ...
    MOV AX, nilai1
    ADD AX, nilai2      ; hasil tambah
    MOV BX, AX          ; simpan agar AX bebas
    MOV AX, nilai1
    SUB AX, nilai2      ; hasil kurang
    JNS CETAK_KURANG
    NEG AX              ; jadikan nilai mutlak
CETAK_KURANG:
    CALL CETAK_AX       ; cetak AX desimal
```

Keluaran

```
Masukkan angka pertama (0-9): 3
Masukkan angka kedua   (0-9): 8
Hasil tambah : 11
Hasil kurang : -5
```

Hasil tambah disimpan di `BX` karena `AX` akan dipakai prosedur pencetakan. Prosedur `CETAK_AX` membagi AX dengan 10 berulang kali dan menyimpan sisanya pada *stack*.

`aritmatika.asm`

Contoh 2: Perkalian, Pembagian, dan *Divide Overflow*

```
MOV AL, 25       ; pengali 8 bit
MOV BL, 4
MUL BL           ; AX = AL x BL = 100

MOV AL, 0FDh     ; 0FDh = -3
MOV BL, 5
IMUL BL          ; AX = -15 (0FFF1h)
...
MOV AX, 1000     ; bilangan yang dibagi
MOV BL, 7        ; pembagi
CMP AH, BL       ; luapan bila AH >= BL
JAE LUAPAN       ; periksa sebelum DIV
DIV BL           ; AL = 142, AH = 6
```

Keluaran

```
MUL  : 25 x 4   = 100
IMUL : -3 x 5   = -15
DIV  : 1000 / 7 = 142  sisa 6
DIV  : 1000 / 3 = divide
overflow: hasil bagi tidak
muat pada AL
```

$1000 \div 3 = 333$ tidak muat pada `AL` yang hanya menampung 0–255. Tanpa pemeriksaan, program berhenti mendadak; kesalahan ini tidak terdeteksi saat perakitan.

`kali-bagi.asm`

Contoh 3: Penjumlahan 32 Bit dengan ADC

```
rendahA DW 0CDEFh    ; word rendah A
tinggiA DW 89ABh     ; word tinggi A
rendahB DW 4567h     ; word rendah B
tinggiB DW 0123h     ; word tinggi B
...
MOV AX, rendahA      ; AX = CDEFh
ADD AX, rendahB      ; AX = 1356h, CF = 1
MOV hasilL, AX       ; simpan word rendah

MOV AX, tinggiA      ; AX = 89ABh
ADC AX, tinggiB      ; AX = 8ACFh
MOV hasilT, AX       ; simpan word tinggi

MOV AH, 4Ch
INT 21h
```

Yang diperlihatkan

ADD pada tahap pertama menghasilkan word rendah sekaligus meninggalkan CF = 1. Nilai inilah yang "dibawa" ke tahap kedua oleh ADC.

Hasil akhirnya 8ACF1356h. Pola yang sama berlaku untuk pengurangan 32 bit dengan pasangan SUB dan SBB.

Penerapan langsung dari tabel propagasi *carry*.

Praktikum: Membangun Kalkulator Sederhana

Judul praktikum

Membangun kalkulator sederhana (masukan–proses–keluaran) ▪ **Sub-CPMK:** 3.1 ▪ **Waktu:** 100 menit ▪ **Bentuk:** individu ▪ **Tugas 4** dikumpulkan

Alat dan bahan

- Komputer dengan Emu8086, GUI TASM, atau DOSBox
- Berkas `aritmatika.asm` dan `kali-bagi.asm`
- Lembar kerja Tugas 4 dan jurnal praktikum

Langkah kerja

1. Rakit dan jalankan kedua contoh; catat keluarannya.
2. Amati `AX`, `BX`, `CF`, `ZF`, `OF`, dan `SP` dengan *Single Step*.
3. Hapus sementara `CMP AH, BL`, lalu amati $1000 \div 3$.
4. Salin kerangka menjadi `kalkulator.asm` dan tambahkan menu empat operasi.

Dikumpulkan: `kalkulator.asm`, tangkapan layar keempat operasi dan pesan pembagi nol, tabel pengamatan *flag*, serta jurnal praktikum.

Latihan

1. Jelaskan tiga aturan legalitas `MOV`. Untuk setiap aturan, berikan satu contoh yang sah dan satu contoh yang ditolak assembler.
2. Apakah perbedaan hasil antara `LEA BX, pesan` dan `MOV BX, pesan`? Kapan `LEA` lebih tepat dipakai daripada `MOV` dengan `OFFSET`?
3. Diberikan `MOV AL, 250` lalu `ADD AL, 15`. Berapa nilai akhir `AL` dalam heksadesimal, dan berapa nilai `CF`, `ZF`, `SF`, serta `OF`?
4. Sebuah program menjalankan `DIV BL` dengan `AX` $=$ `0BB8h` dan `BL` $=$ `03h`, lalu berhenti mendadak. Jelaskan penyebabnya dan tuliskan pemeriksaan yang perlu ditambahkan.
5. Tulislah program yang menjumlahkan $A =$ `0001FFFFh` dan $B =$ `00020001h` dengan `ADD` dan `ADC`. Tunjukkan word rendah, word tinggi, dan `CF` setiap tahap.

Rangkuman

- `MOV` menyalin tanpa menghapus sumber; legalitasnya dibatasi ukuran operand yang sama, larangan memori ke memori, dan larangan nilai segera ke register segmen.
- Pemindahan yang ditolak diatasi dengan register perantara. `MOV` memindahkan isi memori, `LEA` memindahkan alamatnya; `XCHG` menukar isi dua operand.
- `PUSH` dan `POP` bekerja pada satuan word, menolak nilai segera, dan harus berpasangan seimbang agar `SP` kembali.
- Instruksi aritmatika dasar meninggalkan jejak pada *flag*, kecuali `CF` yang tidak disentuh `INC` dan `DEC`. `CF` untuk operasi tak bertanda, `OF` untuk bertanda.
- `MUL`/`IMUL` memakai operand implisit `AL` atau `AX` dan menghasilkan dua kali ukuran operand; `DIV`/`IDIV` menuntut pemeriksaan `CMP AH, BL`.
- Aritmatika bertingkat disusun dengan `ADC` dan `SBB`; bilangan biner diubah menjadi digit desimal dengan pembagian berulang sepuluh.

Cek Pemahaman

1. Instruksi berikut yang *tidak* sah pada prosesor 8086 adalah ...

   1.1 `MOV AX, BX`
   1.2 `MOV AX, nilai1`
   1.3 `MOV nilai1, nilai2`
   1.4 `MOV nilai1, AX`

2. Instruksi yang *tidak* mengubah nilai `CF` adalah ...

   2.1 `ADD AX, BX`
   2.2 `SUB AX, BX`
   2.3 `INC AX`
   2.4 `NEG AX`

3. Kesalahan *divide overflow* pada `DIV BL` dapat dicegah dengan memeriksa ...

   3.1 apakah `AL` bernilai nol
   3.2 apakah `AH` lebih kecil daripada `BL`
   3.3 apakah `ZF` bernilai satu
   3.4 apakah `BL` lebih kecil daripada `AL`

Sebelum pertemuan berikutnya

- Selesaikan Tugas 4: `kalkulator.asm` beserta tangkapan layar keempat operasi dan pesan pembagi nol.
- Lengkapi tabel pengamatan `AX`, `CF`, `ZF`, dan `OF` serta jurnal praktikum.
- Ulangi bagian yang belum dikuasai dengan menelusuri `kali-bagi.asm` satu instruksi demi satu.
- Baca **Bab XI** — Operasi Logika, Pergeseran, dan Rotasi.

Bacaan lanjutan: Bab X bagian Operand Implisit, Makna Flag Hasil Aritmatika, dan Kesalahan Divide Overflow; aturan *flag* rinci pada dokumentasi Intel 8086; pembahasan interupsi dan layanan DOS pada Bab XII.

Terima kasih

Bahasa Pemrograman Tingkat Rendah (MWCA010) ▪ Semester Ganjil